

Relatório de Teste Estrutural e Testes Unitários com JUnit

Integrantes: Matheus Goulart de Alencar e Miguel Falcão Costacurta

Este relatório reúne os resultados do projeto introdutório de boletim e do laboratório Central de pedidos. Os códigos-fonte e testes JUnit acompanham este PDF na mesma pasta.

1 Boletim simples

Resultado

26 execuções aprovadas, zero falhas/erros/ignorados. Foram preservados os quatro testes presentes no arquivo original e acrescentadas 22 execuções em `CoberturaBoletimTest`. O README original descreve apenas um exemplo, mas o código baixado já contém quatro; os números abaixo provêm da execução real.

Executar `mvn clean test`. O comando executa os testes e gera o relatório JaCoCo em `target/site/jacoco/index.html`.

Casos e resultados

Método	Entradas	Esperado	Critério
calcularMedia	(0,0), (10,10), (5,8), (8,5), (0,10), (3.1,4.2)	0; 10; 6.5; 6.5; 5; 3.65	extremos, decimal e ordem das parcelas; tolerância 0.0001
verificarSituacao	0; 3.99; 4; 4.01; 6.99; 7; 7.01; 10	REPROVADO; REPROVADO; RECUPERACAO; RECUPERACAO; RECUPERACAO; APROVADO; APROVADO; APROVADO	abaixo, igual e acima dos limites 4 e 7
contarAprovados	[]; [7]; [6.99]; [8,5,7,0,10]	0; 1; 0; 3	laço com zero, uma e várias iterações; dois ramos
calcularPontos	(false,false), (true,false), (false,true), (true,true)	0; 2; 1; 3	quatro combinações de decisões independentes

Somente notas de 0 a 10 e arrays não nulos, conforme escopo. Nenhuma regra de produção foi alterada.

Cobertura

Etapa	Execuções	Linhas	Ramos	Métodos	Classes
Inicial	4	9/21 (42,86%)	4/12 (33,33%)	3/6 (50,00%)	1/2 (50,00%)
Final	26	21/21 (100,00%)	12/12 (100,00%)	6/6 (100,00%)	2/2 (100,00%)

- **Linhas:** instruções dessas linhas foram exercitadas. O final cobre 21/21.
- **Ramos:** alternativas verdadeira/falsa de cada decisão foram executadas; 12/12 no final.
- **Métodos:** foram chamados os quatro métodos de negócio e os dois construtores; 6/6.
- **Classes:** pelo menos um método de cada classe foi chamado; 2/2. Esse percentual sozinho não garante os demais.

Caminhos e complexidade

`calcularMedia` não tem decisão, $V=1$. `verificarSituacao` tem dois ifs, $V=3$: aprovado (7), recuperação (4), reprovado (3.99). `contarAprovados` tem for e if, $V=3$: vazio, uma nota aprovada e uma nota não aprovada formam uma base. Várias iterações completam o exercício.

`calcularPontos` também tem $V=3$, pois são duas decisões. Base possível: false/false, true/false, false/true. A combinação true/true é um quarto caminho possível que não aumenta a base e verifica a soma dos dois benefícios. Apenas true/true e false/false já cobrem todos os ramos, mas deixam as outras duas combinações sem execução. As asserções verificam o resultado; a cobertura apenas registra execução.

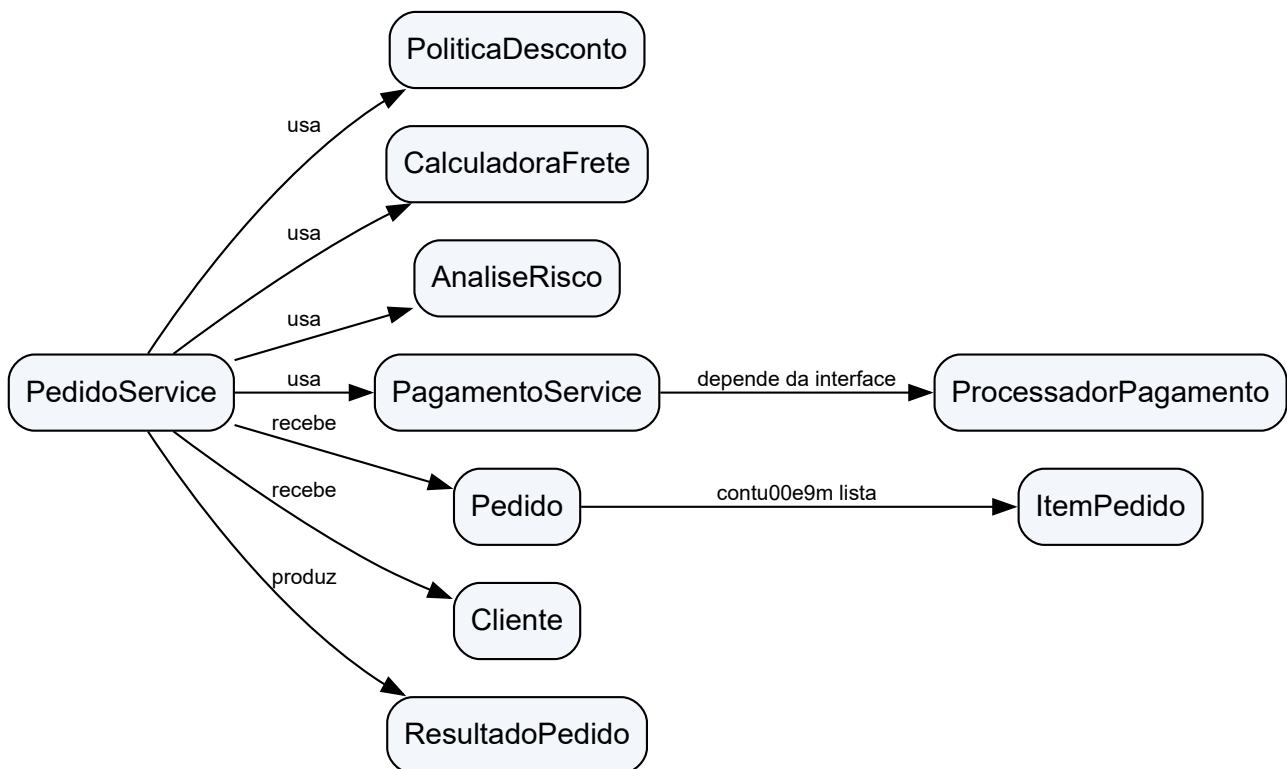
2 Central de pedidos

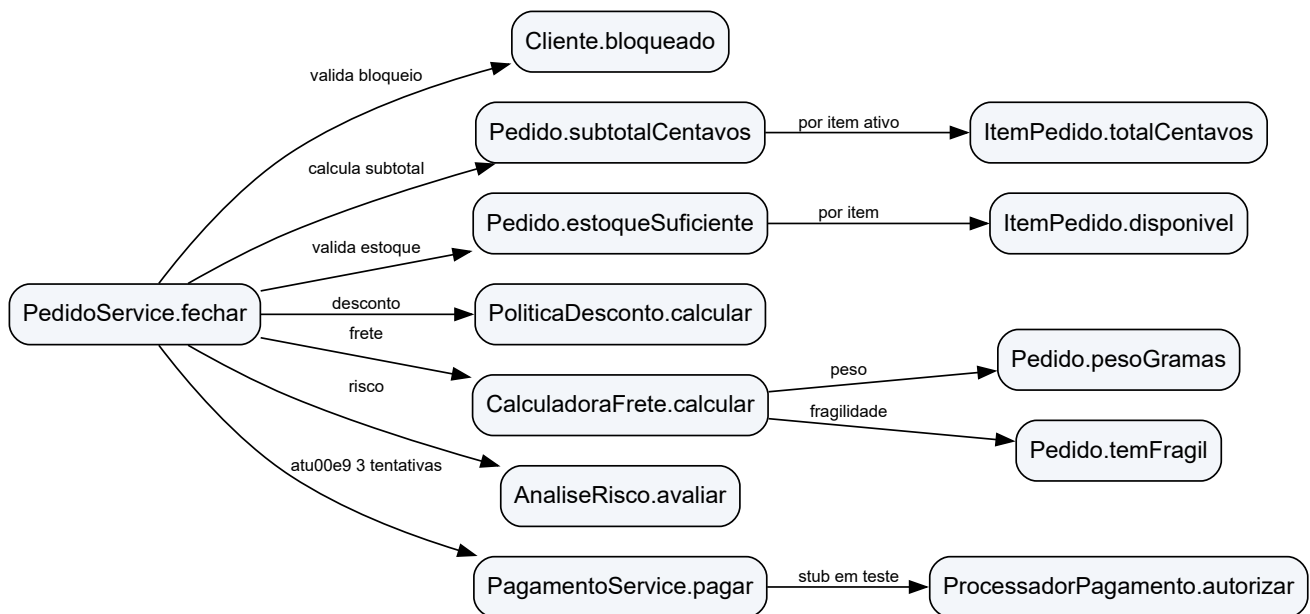
Resultado e reprodução

109 execuções JUnit aprovadas, sem falhas, erros ou testes ignorados. As regras de produção foram preservadas. Ambiente da execução registrada: Windows, OpenJDK 21.0.11, Maven 3.9.16, alvo Java 17, JUnit 5.11.4 e JaCoCo 0.8.13. Executar `mvn clean test` nesta pasta. O comando executa os testes e gera o relatório JaCoCo em `target/site/jacoco/index.html`.

Grafos e complexidade

O grafo de classes apresenta dependências; o grafo de chamadas detalha a colaboração a partir de `fechar`. Os grafos de fluxo de controle seguintes representam o controle interno de cada método.





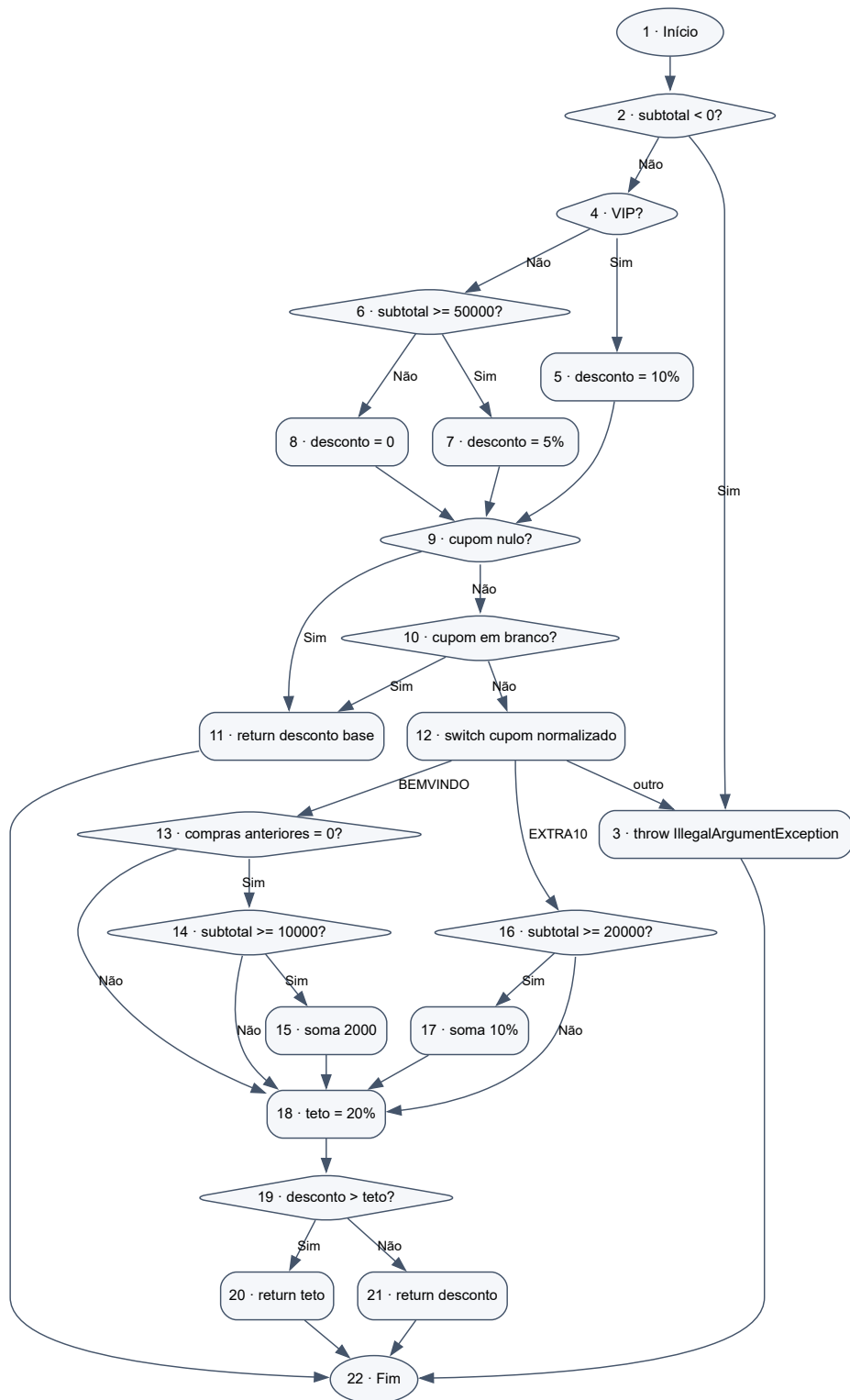
Convenções

- Entrada A e saída unificada Z, incluindo retornos e `throw` explícitos.
- Cada operando de curto-circuito (`&&` e `||`) recebe decisão própria. Expressões aritméticas e chamadas sem desvio explícito ficam em blocos de processamento.
- Em `calcular` do frete, PR, SP, RJ e demais UFs são alternativas distintas do switch. SP e RJ foram separados para documentar dados dos dois rótulos, embora compartilhem o mesmo bloco no bytecode. Essa expansão acrescenta 1 à complexidade em relação ao JaCoCo.
- Em `pagar`, a chamada ao processador tem três saídas: retorno normal, exceção temporária tratada e outra exceção propagada. O retorno booleano normal é devolvido diretamente, sem inventar um `if`. As duas arestas excepcionais acrescentam 2 à complexidade em relação ao JaCoCo, que não conta tratamento de exceções como branch.
- Nos outros grafos de fluxo de controle, exceções implícitas de bibliotecas/chamadas são anotadas no texto e testadas quando contratadas, sem adicionar uma aresta para toda instrução que hipoteticamente possa lançar erro. Em `fechar`, `requireNonNull` é uma chamada no bloco 1; referências nulas terminam excepcionalmente e têm testes próprios.
- A fórmula geral é $E - N + 2$. Em switch ou chamada com três saídas, usar a soma de $(saídas - 1) + 1$, não apenas contar o desenho como uma decisão binária.

Método	Nó s	Aresta s	V(G)	Base viável	JaCoCo Cxty	Restrição relevante
PoliticaDesconto.calcular	22	32	12	12	12	Cupom nulo/branco retorna antes do switch; teto depende das regras anteriores.
CalculadoraFrete.calcular	22	31	11	11	10	Entrega normal e expressa são complementares; considerar líquido antes da gratuidade.
AnaliseRisco.avaliar	13	19	8	8	8	Cliente novo e antigo seguem subárvores distintas.
PagamentoService.pagar	14	19	7	7	5	Limite entre 1 e 3; somente falha temporária repete.
PedidoService.fechar	17	21	6	6	6	RECUSADO por bloqueio é interceptado antes da análise de risco.

As sequências abaixo foram verificadas contra as arestas dos desenhos. A independência foi conferida por eliminação dos vetores de incidência das arestas sobre $GF(2)$, obtendo $V(G)$ vetores independentes em cada grafo. Isso é um certificado de independência, não evidência de que todas as combinações de dados foram testadas. Em caminhos com laços, repetições adicionais não criam automaticamente novas dimensões.

PoliticaDesconto.calcular



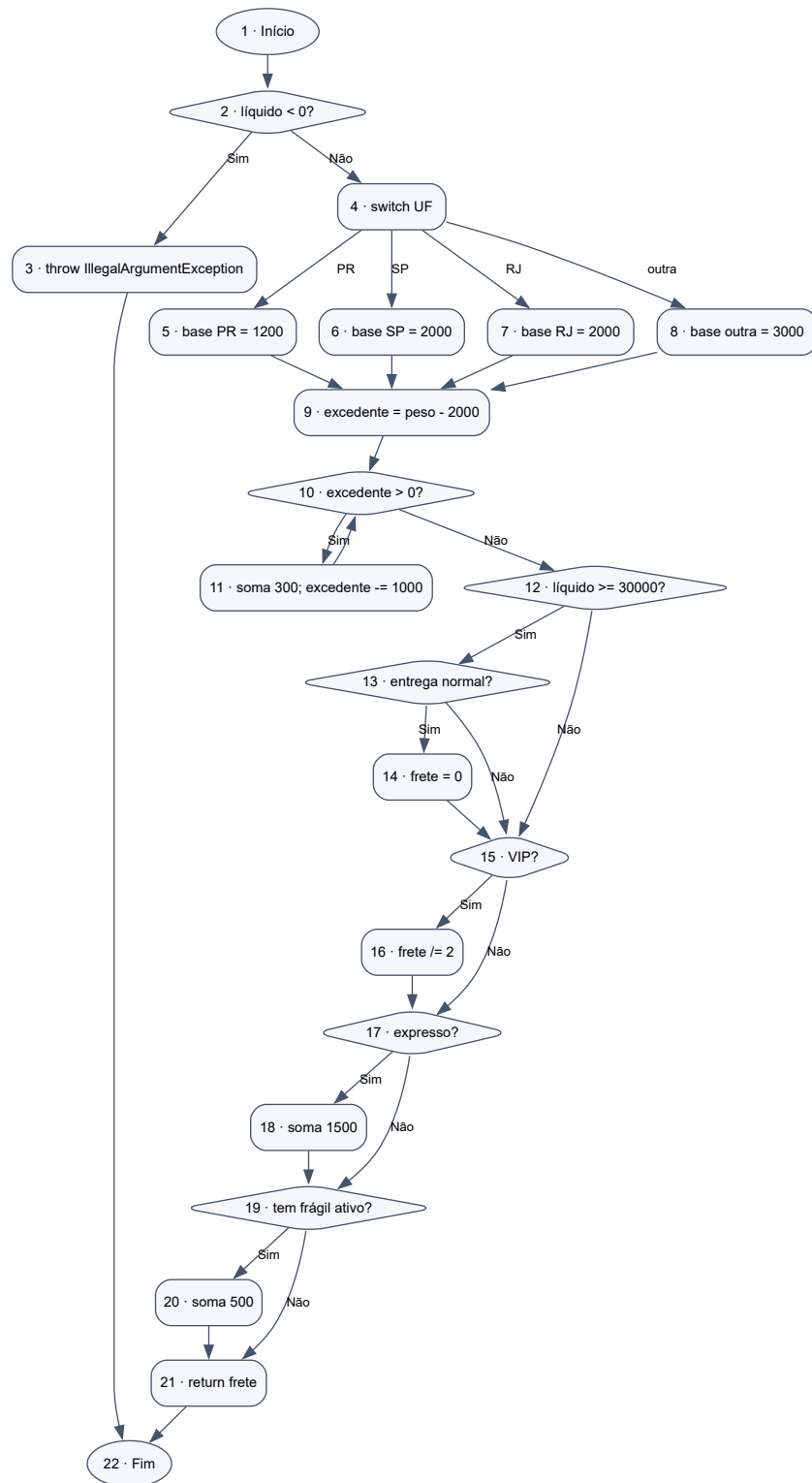
$N = 22$; $E = 32$; $V(G) = 32 - 22 + 2 = 12$.

Caminho	Sequência de nós	Entradas	Resultado esperado
Caminho 1	1 → 2 → 3 → 22	subtotal=-1; VIP=False; compras=0; cupom=None	IllegalArgumentException
Caminho 2	1 → 2 → 4 → 6 → 8 → 9 → 11 → 22	subtotal=0; VIP=False; compras=0; cupom=None	0
Caminho 3	1 → 2 → 4 → 6 → 8 → 9 → 10 → 11 → 22	subtotal=0; VIP=False; compras=0; cupom=' '	0

Caminho	Sequência de nós	Entradas	Resultado esperado
Caminho 4	1 → 2 → 4 → 6 → 8 → 9 → 10 → 12 → 13 → 14 → 18 → 19 → 21 → 22	subtotal=0; VIP=False; compras=0; cupom='BEMVINDO'	0
Caminho 5	1 → 2 → 4 → 6 → 8 → 9 → 10 → 12 → 16 → 18 → 19 → 21 → 22	subtotal=0; VIP=False; compras=0; cupom='EXTRA10'	0
Caminho 6	1 → 2 → 4 → 6 → 8 → 9 → 10 → 12 → 3 → 22	subtotal=0; VIP=False; compras=0; cupom='X'	IllegalArgumentExce ption
Caminho 7	1 → 2 → 4 → 6 → 8 → 9 → 10 → 12 → 13 → 18 → 19 → 21 → 22	subtotal=0; VIP=False; compras=1; cupom='BEMVINDO'	0
Caminho 8	1 → 2 → 4 → 5 → 9 → 11 → 22	subtotal=0; VIP=True; compras=0; cupom=None	0
Caminho 9	1 → 2 → 4 → 6 → 8 → 9 → 10 → 12 → 13 → 14 → 15 → 18 → 19 → 21 → 22	subtotal=10000; VIP=False; compras=0; cupom='BEMVINDO'	2000
Caminho 10	1 → 2 → 4 → 5 → 9 → 10 → 12 → 13 → 14 → 15 → 18 → 19 → 20 → 22	subtotal=10000; VIP=True; compras=0; cupom='BEMVINDO'	2000
Caminho 11	1 → 2 → 4 → 6 → 8 → 9 → 10 → 12 → 16 → 17 → 18 → 19 → 21 → 22	subtotal=20000; VIP=False; compras=0; cupom='EXTRA10'	2000
Caminho 12	1 → 2 → 4 → 6 → 7 → 9 → 11 → 22	subtotal=50000; VIP=False; compras=0; cupom=None	2500

Foram identificados 12 caminhos viáveis linearmente independentes.

CalculadoraFrete.calcular



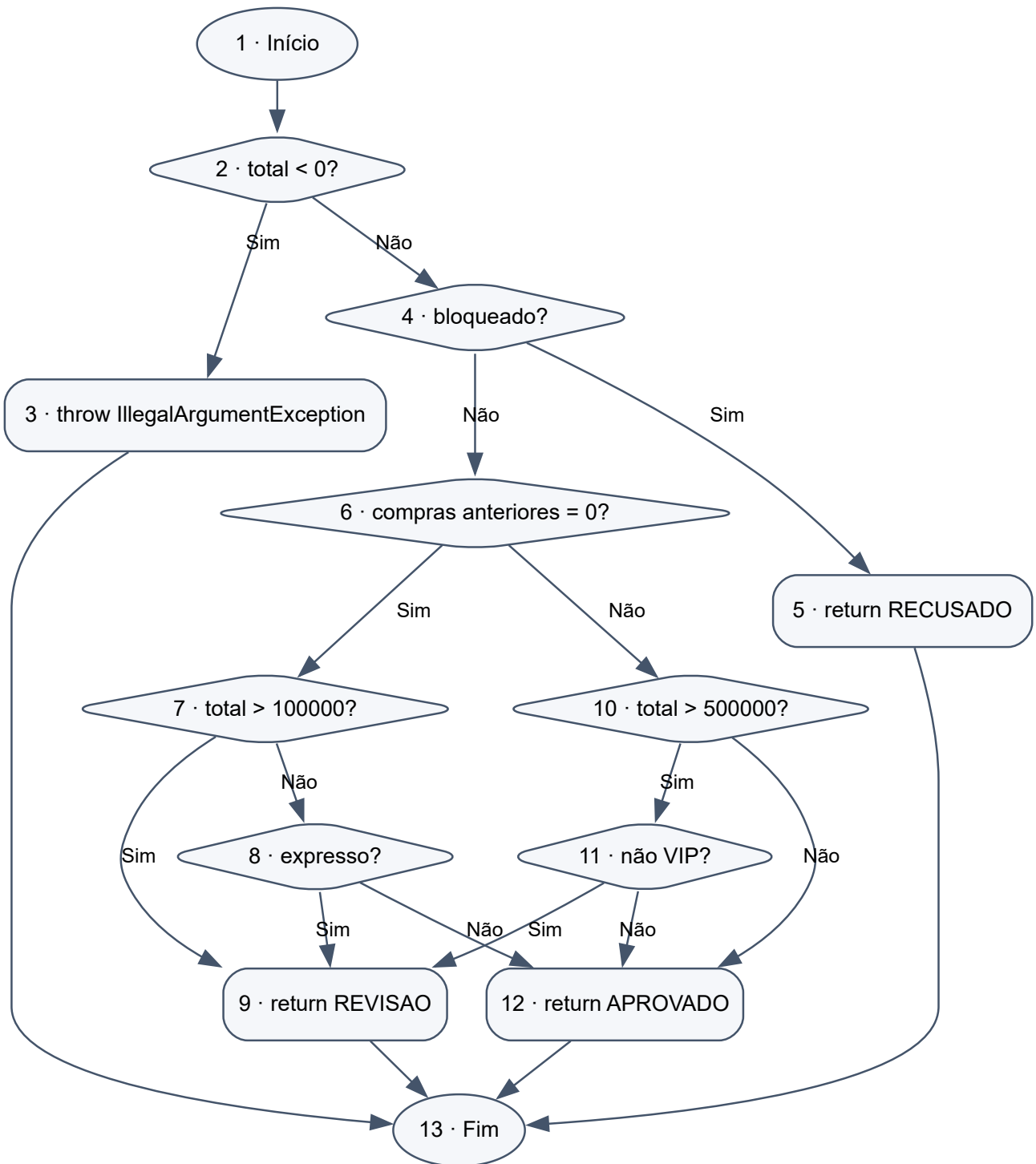
$N = 22$; $E = 31$; $V(G) = 31 - 22 + 2 = 11$.

Caminho	Sequência de nós	Entradas	Resultado esperado
Caminho 1	1 → 2 → 3 → 22	líquido=-1; UF=PR; peso=2000; VIP=False; expresso=False; frágil=False	IllegalArgumentException
Caminho 2	1 → 2 → 4 → 5 → 9 → 10 → 12 → 15 → 17 → 19 → 21 → 22	líquido=100; UF=PR; peso=2000; VIP=False; expresso=False; frágil=False	1200
Caminho 3	1 → 2 → 4 → 5 → 9 → 10 → 12 → 15 → 17 → 19 → 20 → 21 → 22	líquido=100; UF=PR; peso=2000; VIP=False; expresso=False; frágil=True	1700

Caminho	Sequência de nós	Entradas	Resultado esperado
Caminho 4	1 → 2 → 4 → 5 → 9 → 10 → 12 → 15 → 17 → 18 → 19 → 21 → 22	líquido=100; UF=PR; peso=2000; VIP=False; expresso=True; frágil=False	2700
Caminho 5	1 → 2 → 4 → 5 → 9 → 10 → 12 → 15 → 16 → 17 → 19 → 21 → 22	líquido=100; UF=PR; peso=2000; VIP=True; expresso=False; frágil=False	600
Caminho 6	1 → 2 → 4 → 5 → 9 → 10 → 11 → 10 → 12 → 15 → 17 → 19 → 21 → 22	líquido=100; UF=PR; peso=2001; VIP=False; expresso=False; frágil=False	1500
Caminho 7	1 → 2 → 4 → 6 → 9 → 10 → 12 → 15 → 17 → 19 → 21 → 22	líquido=100; UF=SP; peso=2000; VIP=False; expresso=False; frágil=False	2000
Caminho 8	1 → 2 → 4 → 7 → 9 → 10 → 12 → 15 → 17 → 19 → 21 → 22	líquido=100; UF=RJ; peso=2000; VIP=False; expresso=False; frágil=False	2000
Caminho 9	1 → 2 → 4 → 8 → 9 → 10 → 12 → 15 → 17 → 19 → 21 → 22	líquido=100; UF=ZZ; peso=2000; VIP=False; expresso=False; frágil=False	3000
Caminho 10	1 → 2 → 4 → 5 → 9 → 10 → 12 → 13 → 14 → 15 → 17 → 19 → 21 → 22	líquido=30000; UF=PR; peso=2000; VIP=False; expresso=False; frágil=False	0
Caminho 11	1 → 2 → 4 → 5 → 9 → 10 → 12 → 13 → 15 → 17 → 18 → 19 → 21 → 22	líquido=30000; UF=PR; peso=2000; VIP=False; expresso=True; frágil=False	2700

Foram identificados 11 caminhos viáveis linearmente independentes.

AnaliseRisco.avaliar



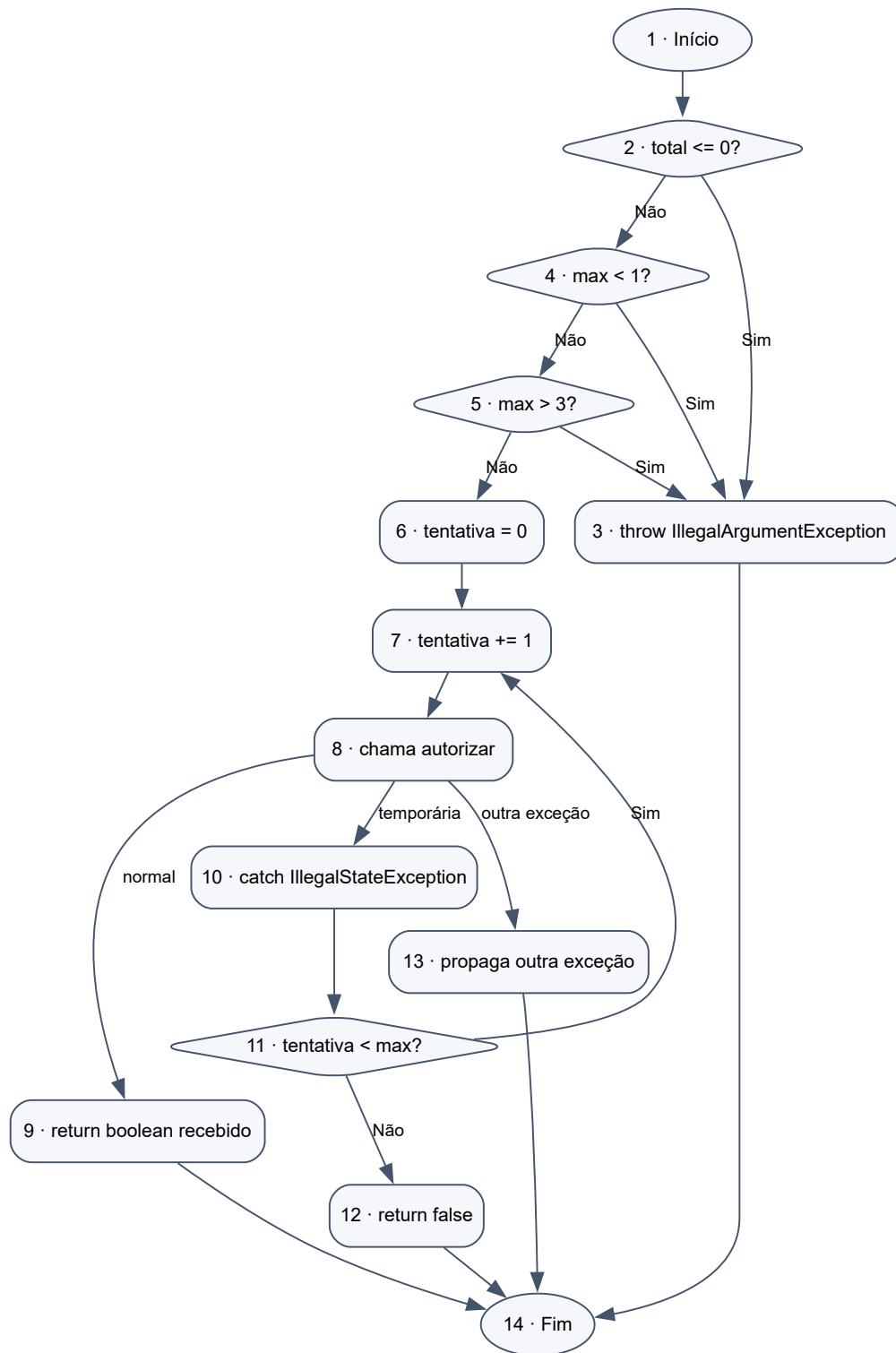
$N = 13$; $E = 19$; $V(G) = 19 - 13 + 2 = 8$.

Caminho	Sequência de nós	Entradas	Resultado esperado
Caminho 1	1 → 2 → 3 → 13	total=-1; bloqueado=False; compras=0; expresso=False; VIP=False	IllegalArgumentException
Caminho 2	1 → 2 → 4 → 6 → 7 → 8 → 12 → 13	total=0; bloqueado=False; compras=0; expresso=False; VIP=False	APROVADO
Caminho 3	1 → 2 → 4 → 6 → 7 → 8 → 9 → 13	total=0; bloqueado=False; compras=0; expresso=True; VIP=False	REVISAO
Caminho 4	1 → 2 → 4 → 6 → 10 → 12 → 13	total=0; bloqueado=False; compras=1; expresso=False; VIP=False	APROVADO
Caminho 5	1 → 2 → 4 → 5 → 13	total=0; bloqueado=True; compras=0; expresso=False; VIP=False	RECUSADO
Caminho 6	1 → 2 → 4 → 6 → 7 → 9 → 13	total=100001; bloqueado=False; compras=0; expresso=False; VIP=False	REVISAO

Caminho	Sequência de nós	Entradas	Resultado esperado
Caminho 7	1 → 2 → 4 → 6 → 10 → 11 → 9 → 13	total=500001; bloqueado=False; compras=1; expresso=False; VIP=False	REVISAO
Caminho 8	1 → 2 → 4 → 6 → 10 → 11 → 12 → 13	total=500001; bloqueado=False; compras=1; expresso=False; VIP=True	APROVADO

Foram identificados 8 caminhos viáveis linearmente independentes.

PagamentoService.pagar

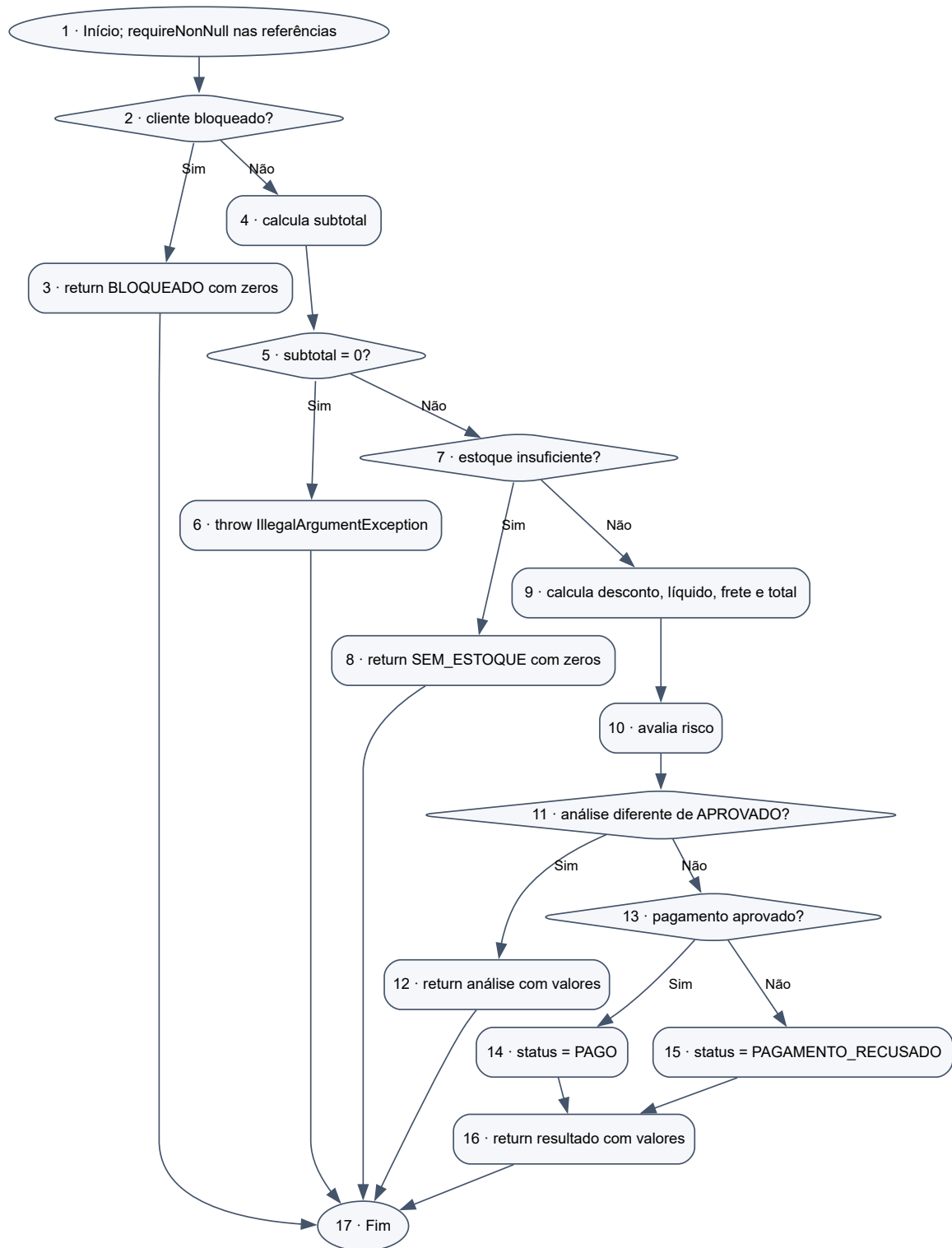


$$N = 14; E = 19; V(G) = 19 - 14 + 2 = 7.$$

Caminho	Sequência de nós	Entradas	Resultado esperado
Caminho 1	1 → 2 → 3 → 14	total=0; max=3	IllegalArgumentException
Caminho 2	1 → 2 → 4 → 3 → 14	total=1; max=0	IllegalArgumentException
Caminho 3	1 → 2 → 4 → 5 → 3 → 14	total=1; max=4	IllegalArgumentException
Caminho 4	1 → 2 → 4 → 5 → 6 → 7 → 8 → 9 → 14	total=100; max=3; stub=true	true, 1 chamada
Caminho 5	1 → 2 → 4 → 5 → 6 → 7 → 8 → 13 → 14	total=100; max=3; stub=UnsupportedOperationException	propaga; 1 chamada
Caminho 6	1 → 2 → 4 → 5 → 6 → 7 → 8 → 10 → 11 → 12 → 14	total=100; max=1; stub=IllegalStateException	false; 1 chamada
Caminho 7	1 → 2 → 4 → 5 → 6 → 7 → 8 → 10 → 11 → 7 → 8 → 9 → 14	total=100; max=2; stub=IllegalStateException, true	true; 2 chamadas

Foram identificados 7 caminhos viáveis linearmente independentes.

PedidoService.fechar



$N = 17$; $E = 21$; $V(G) = 21 - 17 + 2 = 6$.

Caminho	Sequência de nós	Entradas	Resultado esperado
Caminho 1	1 → 2 → 3 → 17	bloqueado=true; lista vazia; cupom inválido	BLOQUEADO; zeros; 0 cobranças
Caminho 2	1 → 2 → 4 → 5 → 6 → 17	desbloqueado; lista vazia	IllegalArgumentException; 0 cobranças
Caminho 3	1 → 2 → 4 → 5 → 7 → 8 → 17	item 10000; quantidade 1; estoque 0; cupom inválido	SEM_ESTOQUE; zeros; 0 cobranças

Caminho	Sequência de nós	Entradas	Resultado esperado
Caminho 4	1 → 2 → 4 → 5 → 7 → 9 → 10 → 11 → 12 → 17	novo cliente comum; item 10000; PR; expresso	REVISAO; 10000, 0, 2700, 12700; 0 cobranças
Caminho 5	1 → 2 → 4 → 5 → 7 → 9 → 10 → 11 → 13 → 14 → 16 → 17	cliente comum antigo; item 10000; PR; normal; stub=true	PAGO; 10000, 0, 1200, 11200; 1 cobrança
Caminho 6	1 → 2 → 4 → 5 → 7 → 9 → 10 → 11 → 13 → 15 → 16 → 17	mesmos dados; stub=false	PAGAMENTO_RECUSADO; mesmos valores; 1 cobrança

Foram identificados 6 caminhos viáveis linearmente independentes.

Matriz de testes e contribuição para a cobertura

Valores monetários em centavos. Os testes parametrizados representam várias execuções; as tabelas literais dos arquivos de teste fornecem cada entrada e esperado. `CaminhosIndependentesTest` contém uma execução com o mesmo identificador Pn de cada base de desconto, frete e risco.

ID ou método JUnit	Unidade e entrada/estado	Resultado esperado	Caminho ou critério acrescentado
<code>PedidoServiceTest.deveFecharPedidoDeClienteComumComFreteDoParanaEPagamentoAprova do</code>	comum, PR, item disponível, stub aprova	total 11200 e uma chamada	exemplo inicial; caminho de sucesso
<code>DominioCompletoTest.cliente</code>	histórico -1 e 0; atributos true	exceção para -1; getters corretos	validação do histórico
<code>limitesItem</code>	mínimos e máximos de preço, quantidade e peso; estoque insuficiente	totais 0 e 100000000; disponibilidade correta	fronteiras e dois ramos de disponibilidade
<code>itensInvalidos</code>	SKU nulo/branco; preços 0,-1,1000001; quantidades -1,101; estoque -1; pesos 0,-1,100001	<code>IllegalArgumentException</code>	cada validação e curto-circuito
<code>pedidoInvalido</code>	lista nula, 101 itens, elemento nulo, UF malformada; ZZ	exceções especificadas; ZZ aceito	validações de lista/UF e default válido
<code>vazio</code>	lista vazia	subtotal/peso 0; frágil false; estoque true	zero iterações
<code>copiaDefensiva</code>	alterar lista original e lista retornada	pedido preservado; lista imutável	contrato de imutabilidade
<code>ativosInativosERepetidos</code>	inativo frágil + ativos 1 e 2	subtotal 300, peso 3000, frágil true, estoque true	continue, curto-circuito de fragilidade, múltiplas linhas
<code>estoqueInicioEFim</code>	falta no primeiro / último item	false	break em posições distintas
<code>dominioMaximo</code>	100 linhas × 100 unidades nos máximos	subtotal 10000000000 e peso 1000000000	limites agregados e tipos numéricos
<code>resultado</code>	status e quatro valores	getters iguais à construção	objeto resultado
<code>DescontoCompletoTest.regras</code>	tabela VIP/comum, 49999/50000/50001, 9999/10000/10001, 19999/20000/20001, cupons	descontos literais de 0 a 7500 na tabela	todos os ramos, teto e truncamento
<code>normalizacao / invalido</code>	espaços e minúsculas; negativo; desconhecido	2000 / <code>IllegalArgumentException</code>	normalização e erros
<code>FreteCompletoTest.tarifas</code>	PR/SP/RJ/ZZ, 1999/2000/2001/3000/3001 g, líquido 29999/30000/30001; combinações	tabela literal; VIP+expresso+frágil = 2600	switch, while, fronteiras e decisões independentes
<code>fragilidadeUmaVez / negativo</code>	dois itens frágeis; líquido -1	1700 / <code>IllegalArgumentException</code>	adicional único; erro
<code>RiscoCompletoTest.regras / negativo</code>	novo/antigo, bloqueado, VIP, expresso e limites 100000/500000	RECUSADO, REVISAO ou APROVADO; erro para -1	curto-circuitos OR e AND; limites
<code>PagamentoCompletoTest.aprovaUmaVez / recusaSemRepetir</code>	stub true ou false	boolean correspondente; uma chamada de 123	retorno imediato, sem repetição por recusa
<code>sucessoAposFalhas</code>	sucesso nas tentativas 1,2,3	true; contador 1,2,3; total 100	aresta de retorno do do/while
<code>esgota</code>	falha temporária sempre; max 1,2,3	false; contador igual ao limite	término do laço

ID ou método JUnit	Unidade e entrada/estado	Resultado esperado	Caminho ou critério acrescentado
falhaDepoisRecusa	IllegalStateException e false	false; duas chamadas	recusa após reentrada
propaga	UnsupportedOperationException	mesma exceção; uma chamada	exceção não capturada, fora do contador de branches
validacoes	processador nulo; total -1/0; max 0/4	exceções; nenhuma chamada	guardas anteriores ao laço
ServicoCompletoTest.nulos	serviço, pedido ou cliente nulo	NullPointerException	contrato de referências
bloqueioAntesDoSubtotalECupom	bloqueado, lista vazia e cupom inválido	BLOQUEADO e zeros; nenhuma cobrança	fechamento Caminho 1 e precedência
semAtivos	vazio / só inativo	IllegalArgumentException; não cobra	fechamento Caminho 2
estoqueAntesDoCupom	falta de estoque e cupom inválido	SEM_ESTOQUE e zeros	fechamento Caminho 3
cupomInvalidoNaoCobra	estoque válido e cupom desconhecido	IllegalArgumentException; não cobra	interrupção antes do pagamento
revisaoNaoCobra	novo, expresso, PR, subtotal 10000	REVISAO; total 12700; não cobra	fechamento Caminho 4
pagoComDescontoEFrete	VIP antigo; 35000; EXTRA10; expresso; PR	desconto 7000, frete 2100, total 30100; uma chamada	colaboração com desconto e frete
recusado	comum; 10000; PR; stub false	PAGAMENTO_RECUSADO; total 11200; uma chamada	fechamento Caminho 6
tresTentativas / indisponivel	duas falhas + true / sempre falha	PAGO / PAGAMENTO_RECUSADO; 3 chamadas	repetição pelo serviço
propagaExcecao	stub UnsupportedOperationException	propaga	colaboração excepcional
CaminhosIndependentesTest.descontoCaminho 1 a Caminho 12	dados iguais à base do grafo	valores/exceções literais	vínculo direto entre base e testes
CaminhosIndependentesTest.freteCaminho1 a Caminho 11	dados iguais à base do grafo	valores/exceções literais	inclui os dois rótulos SP/RJ
CaminhosIndependentesTest.riscoCaminho1 a Caminho 8	dados iguais à base do grafo	classificação/exceção literal	todos os caminhos básicos de risco

Para pagamento, a base Caminho 1–Caminho 3 está em `validacoes`; Caminho 4 em `aprovaUmaVez`; Caminho 5 em `propaga`; Caminho 6 em `esgota(1)`; Caminho 7 em `sucessoAposFalhas(2)` (`max=3` percorre a mesma sequência até o retorno da segunda tentativa). O fechamento Caminho 5 está no exemplo original e em `tresTentativas` quanto ao grafo de fluxo de controle local. As chamadas internas de pagamento possuem seu próprio grafo.

Evolução da cobertura

Etapa	Execuções	Linhas	Ramos	Métodos	Classes
Inicial	1	87/108 (80,56%)	50/116 (43,10%)	20/21 (95,24%)	9/9 (100,00%)
Final	109	108/108 (100,00%)	116/116 (100,00%)	21/21 (100,00%)	9/9 (100,00%)

No estado inicial, as nove classes já eram executadas, mas 66 de 116 ramos não eram cobertos. Os novos testes exercitam erros de validação, cupons, limites, frete, risco e repetição/exceções no pagamento. Cobrir classes não demonstra cobertura de todos os seus métodos ou alternativas.

Alteração proposital e restauração

A tarifa base do PR em `CalculadoraFrete` foi alterada temporariamente de 1200 para 1300. A execução de `FreteCompletoTest` registrou **16 testes, 9 falhas**, incluindo esperado 1200 e obtido 1300. O arquivo de produção foi restaurado byte a byte em bloco `finally`; a execução final passou. Consulte log da mutação e verificação dos arquivos de produção.

Análise crítica

1. **Ramos não equivalem a caminhos:** testar frete com todas as opções falsas e com VIP/expresso/frágil verdadeiros pode cobrir alternativas locais sem testar, por exemplo, frete gratuito com fragilidade. O teste de Líquido 30001, entrega normal e fragilidade verifica que o adicional 500 permanece mesmo com base zero. Combinações adicionais têm valor funcional mesmo quando não elevam cobertura.
2. **Curto-circuito:** cupom nulo não avalia `isBlank`; histórico diferente de zero não avalia o limite BEMVINDO; novo cliente com total acima de 100000 não precisa consultar expresso; cliente antigo com total até 500000 não consulta VIP para revisão. Os testes variam separadamente essas causas.
3. **Caminhos inviáveis:** a combinação cliente novo e cliente antigo na mesma avaliação de risco não existe. Em frete gratuito a condição entrega normal é verdadeira, portanto expresso não pode ser verdadeiro depois. A base usa apenas entradas viáveis. `RECUSADO` do risco é alcançável em teste unitário, mas não via `fechar` com cliente bloqueado, porque o serviço retorna BLOQUEADO antes.
4. **Exceções:** temporária causa repetição; recusa false termina imediatamente; outra exceção propaga. Esses comportamentos foram verificados com contadores e argumentos. JaCoCo não conta arestas excepcionais de try/catch como branches, justificando $V=7$ no modelo de pagamento contra $Cxty=5$ no relatório.
5. **Laços:** pedidos cobrem zero, uma e várias linhas; inativos acionam continue; falta no início/fim aciona break. Peso excedente cobre zero, uma e várias iterações, incluindo kg exato e fração. Pagamento cobre de uma a três tentativas e limites inválidos.
6. **Oráculos:** valores esperados são literais calculados pelas regras do enunciado. Nenhum teste chama a implementação sob teste para produzir seu próprio esperado. $35000-7000+(1200/2+1500)=30100$ é um exemplo de colaboração.
7. **Limitações:** 100% não prova ausência de defeitos, não mede todos os caminhos completos e não garante MC/DC. Testes monetários incluem truncamento e limites, mas o espaço de valores possíveis é muito maior que a suíte.